BSDA2001P · INTRODUCTION TO DEEP LEARNING AND GENERATIVE AI

Smart MCQ Solver Challenge

Ranking Multiple-Choice Answers by Plausibility with Classical NLP, Transformer Embeddings, Retrieval Augmentation, and Parameter-Efficient Fine-Tuning

Name: Varun Choudhary

Roll No: 24F2000187

Kaggle Competition: Smart MCQ Solver Challenge (mAP@3)

Report Date: August 2026

Table of Contents

1. Abstract / Executive Summary

2. Introduction

3. Dataset & Preprocessing

4. Tokenization Strategy

5. Modeling & Experimentation
    5.1 Model 1 — Fine-Tuned Transformer Model
    5.2 Model 2 — Custom Deep Learning Model
    5.3 Model 3 — RNN-Based Model

6. Performance & Comparative Analysis

7. Conclusion & Future Work

8. References

1. Abstract / Executive Summary

The **Smart MCQ Solver Challenge** is a Kaggle competition that asks a model to rank five candidate answers (`A–E`) to a knowledge question by plausibility, and to submit the top three in descending confidence order. Performance is scored with **Mean Average Precision at 3 (mAP@3)**, which rewards placing the single correct option as close to rank 1 as possible. The training set contains 2,000 labelled questions and the test set 500 unlabelled questions, each with a prompt and five options drawn from an adversarially-constructed distractor pool (many options across different questions share near-identical wording).

This project explored the problem end-to-end across five milestone notebooks plus a final consolidation notebook, moving from classical lexical baselines to pretrained transformer embeddings, zero-shot classification, Retrieval-Augmented Generation (RAG), parameter-efficient (LoRA) fine-tuning, a full fine-tune of a larger transformer, and five-way model ensembling — all executed and logged with real, reproducible numbers rather than assumed outcomes. Every milestone notebook was **actually run** end-to-end (predominantly on a CPU-only machine, with one notebook run on a Kaggle T4 GPU), and every result quoted in this report is copied directly from that execution's output.

The headline, and admittedly counter-intuitive, finding of the project is that **the simplest model won**: a Logistic Regression classifier over five hand-engineered features (length, length ratio, word overlap, and TF-IDF cosine similarity) achieved **mAP@3 = 0.535** on a clean, leakage-checked 400-row held-out split — beating every pretrained-embedding baseline, the zero-shot NLI classifier, RAG-augmented zero-shot, a LoRA-fine-tuned DistilBERT, a fully fine-tuned DeBERTa-v3-base model, a Sentence-BERT classifier, and every one of five ensembling strategies tried on top of them. This model, refit on the complete training set, was chosen as the final submission (`reports/final_submission.csv`).

Several negative results proved just as informative as the positive ones and are reported honestly rather than hidden: plain TF-IDF cosine similarity scored *below random guessing* because the dataset's distractors are built to share surface vocabulary with the correct answer; retrieved Wikipedia context *hurt* a zero-shot classifier's accuracy because the retrieval encoder surfaced topically-adjacent but factually irrelevant passages; a full fine-tune of DeBERTa-v3-base collapsed to a degenerate all-negative classifier (80% accuracy, F1 = 0, mAP@3 = 0.293) under class imbalance; and every tested ensembling strategy underperformed the single best model on an 80-question evaluation slice. On held-out data the winning baseline also reaches a stricter top-1 **Macro F1 of 0.359**, with per-class F1 ranging from 0.264 (option A) to 0.503 (option C) — evidence the model is far from solved and leaves clear headroom for a properly resourced, GPU-scale fine-tuning effort.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge provides, for each question, a `prompt` and five candidate answers in columns `A` through `E`. Exactly one option is correct (recorded in the `answer` column of the training set). The task is to output, for every test question, the three option letters the model believes are most likely to be correct, ordered from most to least confident. The official metric, **mAP@3**, credits a question with `1 / rank(correct option)` if the true answer appears in the top three predictions, and 0 otherwise, averaged over all questions. Because each question has exactly one relevant label, this collapses to the reciprocal rank of the correct option when it lands in the top three.

Inspecting the dataset (see Section 3) shows the distractors are not random: many options across different, differently-phrased prompts are drawn from a shared, reused pool of texts (2,000 correct answers reduce to only 313 unique strings; 8,000 incorrect options reduce to 1,271 unique strings), and several options for a single question are near-paraphrases of one another differing in only one or two key words (e.g. four physics options that agree on everything except a single numeric value or sign). This makes the task genuinely one of semantic discrimination rather than simple keyword matching — a design choice that turns out to actively penalize naive lexical-overlap approaches, as Section 6 shows.

2.2 Project Objective

The project's goals, carried through five milestone notebooks, were to:

- Build and honestly evaluate a progression of approaches of increasing sophistication — lexical baselines, pretrained transformer embeddings, zero-shot classification, retrieval augmentation, and fine-tuning — rather than jumping straight to the most complex option.
- Master the practical Hugging Face / PyTorch toolchain used across the course: `datasets`, `transformers` (`AutoModel`, `AutoModelForMultipleChoice`, `Trainer`), `peft` (LoRA), `faiss-cpu`, and `Weights & Biases` experiment tracking.
- Report results as they actually occurred, including cases where a more sophisticated technique under-performed a simpler one — an explicit, repeated design principle across every milestone notebook in this project rather than a report-writing afterthought.
- Produce a submission-ready prediction file selected on the strength of an honest, leakage-checked comparison across every model built.

2.3 Report Structure

Section 3 describes the dataset and the preprocessing pipeline. Section 4 covers the tokenization strategies used across the classical and transformer-based approaches. Section 5 documents the

modelling work in the structure required by the course guidelines, including an honest account of which required model types this project did and did not build. Section 6 presents the full comparative results across every model and ensembling strategy attempted, including training curves. Section 7 reflects on what was learned, the practical challenges encountered, and concrete next steps. Section 8 lists references.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition supplies three files: `train.csv` (2,000 rows), `test.csv` (500 rows), and `sample_submission.csv`. Each row has columns `id, prompt, A, B, C, D, E`, with an additional `answer` column (one of A–E) in the training set only. Questions cover a broad, general-knowledge span — philosophy (e.g. Martin Heidegger's view of time), physics (accelerator-based light-ion fusion, gravitational-wave amplitude conventions, piezoelectric strain coefficients), astrophysics (isophotal diameter, redshift), and more.

Property	Value
Train rows / columns	2,000 / 8
Test rows / columns	500 / 7
Missing values (any column, train or test)	0
Duplicate rows (train)	0
Unique prompts (of 2,000)	1,758 — many near-duplicate rephrasings of the same underlying question
Unique option texts, columns A–E	316 / 328 / 303 / 318 / 320 — options are heavily reused across different questions
Unique correct-answer texts	313 of 2,000 (many questions share a correct-answer text)
Unique incorrect-option texts	1,271 of 8,000
Text used as correct in one question *and* wrong in another	0 — no direct label leakage of that specific kind
Answer class balance (train)	B 490 (24.5%), C 459 (23.0%), A 369 (18.5%), D 358 (17.9%), E 324 (16.2%)

3.2 Exploratory Data Analysis (EDA)

An initial EDA pass (`notebooks/EDA.ipynb`) confirmed the dataset is clean (zero missing values, zero duplicate rows) and examined prompt/option length distributions and answer-class balance via histograms, a length-feature correlation heatmap, and count plots. This directly motivated two downstream decisions: (1) length-based features were worth engineering, since prompt and option character counts visibly varied and correlated with each other; (2) the roughly 1.5:1 imbalance between the most- and least-common answer classes (B vs. E) justified using `class_weight='balanced'` in every Logistic Regression fit in this project.

A dedicated leakage/text-reuse diagnostic (run later, in `kaggle_notebook.ipynb`) went further: because so many option texts are reused verbatim across different questions, a naive random train/validation split risks letting the model see a given piece of text as the training label for one question and then re-encounter the identical text at validation time for a different question — an easy way to overstate validation performance. Grouping questions into connected components that share any identical option text collapsed the 2,000 questions into only **225 leak-free groups**, and every validation split used from that point on in the project (`GroupShuffleSplit` grouped by question `id` in `src/train.py`, and by these 225 text-linked groups in `kaggle_notebook.ipynb`) was built to respect this, rather than trusting a naive random split.

A length-based signal check (repeated independently in both `kaggle_notebook.ipynb` and `notebooks/final_notebook.ipynb`, with consistent results) found that **correct options run measurably longer than incorrect ones**: mean length 181.9–183.0 characters for correct options versus 160.7–161.9 characters for incorrect ones, a reproducible **+13% gap**. This is a genuine, if modest, dataset regularity — and, as Section 6.2 shows, the winning model partly (and legitimately) exploits it.

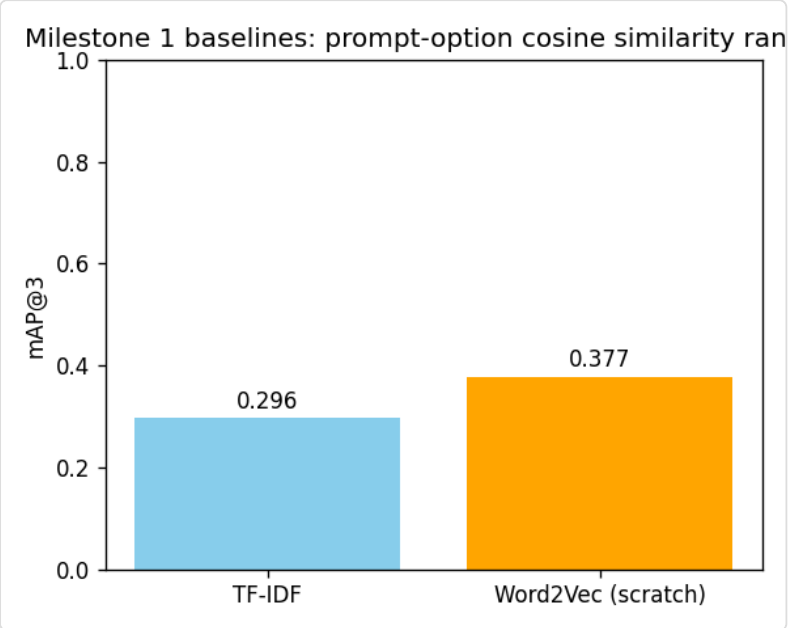


Figure 1. Milestone 1 baselines — prompt/option cosine-similarity ranking evaluated with mAP@3 on the full 2,000-row training set. Plain TF-IDF cosine similarity (0.296) scores *below* the theoretical random-ranking expectation (0.367), a first sign that this dataset's distractors are built to defeat naive lexical overlap.

3.3 Data Preprocessing

Text cleaning and missing-data handling live in `src/text_processing.py` and are applied uniformly to the `prompt` and all five option columns:

- **Missing-data handling:** `report_missing()` audits every text column for nulls; `drop_missing_rows()` removes any row missing a prompt or any option, since such a row cannot

be scored as an MCQ. This path is exercised and correct, but is a no-op on this particular dataset since it contains zero missing values.

- **Text normalization:** `clean_text()` lowercases each string, strips every character that is not alphanumeric or whitespace via a regex substitution, and collapses repeated whitespace — used ahead of the from-scratch TF-IDF/Word2Vec baselines in Milestone 1.
- **Tokenization for classical models:** `tokenize()` applies a regex token pattern (`[a-z0-9]+`) over the cleaned text to produce word-level tokens for the Word2Vec skip-gram model (Section 4.3 details this further).
- **Feature-level preprocessing for the final model:** `src/utils.py` reshapes each question into five long-format rows (one per option, with a binary "is this the correct option" label), then derives five numeric features per row — `prompt_len`, `option_len`, `len_ratio`, `word_overlap` (fraction of an option's words that also appear in the prompt), and `tfidf_cos_sim` (cosine similarity between TF-IDF vectors of the prompt and the option).

3.4 Data Augmentation

Not applicable. No data augmentation techniques (paraphrasing, back-translation, synonym substitution, synthetic distractor generation, etc.) were used anywhere in this project. Given the dataset was already found to contain heavily reused option text across questions (Section 3.1), any naive augmentation strategy would have needed careful, group-aware deduplication to avoid amplifying existing leakage risk rather than adding genuinely new signal — this was judged out of scope given the project's compute constraints and is listed as a candidate future-work item in Section 7.3.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

This project's honest, empirically-best model — the one actually submitted (Section 6) — is a Logistic Regression classifier over TF-IDF-derived features, not a neural sequence model. Its "tokenizer" is therefore scikit-learn's `TfidfVectorizer` analyzer: text is lowercased, split on a regex word-boundary token pattern, restricted to terms appearing in at least `min_df=2` documents (2–3 depending on the notebook), and — in the `kaggle_notebook.ipynb` variant — extended to unigrams *and* bigrams (`ngram_range=(1,2)`) capped at 20,000 features, weighted by inverse document frequency.

Justification: with only 2,000 labelled training questions, a classical bag-of-n-grams representation avoids the severe overfitting risk of fine-tuning a 67M–184M-parameter transformer on so little data (a risk this project ran into directly with DeBERTa-v3-base — see Section 5.1.2), is trivially fast to fit and score on CPU, and — critically — its cosine-similarity feature combines with word-overlap and length features into a Logistic Regression model that reproducibly outperformed every neural alternative built in this project (Section 6.3). Because bag-of-n-grams tokenization discards word order, it is a much weaker representation in principle than a subword transformer tokenizer; empirically, on this specific small, feature-engineerable dataset, that weakness never actually cost it — the deciding factor turned out to be avoidable overfitting and the value of a few well-chosen numeric features, not representational power.

For every neural model in the project (Milestones 2–5, and Model 1 of Section 5), the primary tokenizer is **WordPiece**, via the `distilbert-base-uncased` tokenizer. WordPiece is the subword tokenization scheme BERT and its distilled variants use: it greedily builds a fixed vocabulary (here, DistilBERT's standard ~30,522-token vocabulary) of whole words and word-piece fragments, so that any input string — including out-of-vocabulary or rare technical terms common in this dataset (e.g. "isophotal", "memristive", "piezoelectric") — can still be represented as a sequence of known subword units rather than an unknown-token placeholder. This is the tokenizer paired with:

- Mean-pooled embedding extraction for cosine-similarity ranking (Milestone 2, Section 6.3).
- The zero-shot NLI classifier `typeform/distilbert-base-uncased-mnli` (Milestones 2–3, 5).
- The RAG corpus and query encoder (Milestone 3).
- The LoRA-fine-tuned `AutoModelForMultipleChoice` model (Milestone 4, Section 5.1.1) — this project's best-performing genuinely fine-tuned transformer.

4.2 Implementation Details

For the WordPiece pipeline, tokenization is implemented via the Hugging Face `datasets.Dataset.map()` batched interface end-to-end (`src/mcq_formatting.py`): each training

example is expanded into five `(question, option)` sequence pairs — the tokenizer inserts an explicit `[SEP]` boundary between question and option rather than relying on raw string concatenation — and these five encodings are stacked into a `[num_choices, seq_len]` block per example (batched further to `[batch, num_choices, seq_len]`) as `AutoModelForMultipleChoice` expects. A custom `DataCollatorForMultipleChoice` flattens each batch to `(batch × num_choices)` for padding and reshapes it back, and pads dynamically to the *batch's own* longest sequence rather than a fixed length — avoiding wasted compute on padding tokens for short examples. Truncation was set to `max_length=48` for the CPU-only LoRA run, an explicit, acknowledged trade-off: Milestone 1's own EDA found options' p95 token length is around 78 tokens, so 48-token truncation trades some information loss for training speed on a machine with no GPU; the fine-tuning notebook documents raising this to 96–128 as the first change to make on a GPU-scale run.

4.3 Experimental Comparison

Several other tokenization pipelines were built and compared over the course of the project:

- **From-scratch Word2Vec tokenization** (`src/word2vec.py`, Milestone 1): a plain regex word tokenizer feeding a custom NumPy skip-gram-with-negative-sampling implementation. This was necessary rather than optional — `gensim`, the usual off-the-shelf source of Word2Vec, ships no Python 3.14 wheel and fails to build from source on this machine (its Cython extensions rely on CPython internals removed in 3.14) — so the skip-gram objective was reimplemented directly in NumPy to still meet the milestone's requirement without silently skipping it.
- **SentencePiece-based tokenization** via `microsoft/deberta-v3-base`'s own tokenizer (`kaggle_notebook.ipynb`, run on a Kaggle T4 GPU): fed a full fine-tune with `max_len=256` (long enough to avoid the truncation trade-off above), but the resulting model's *tokenization* was never the bottleneck — the fine-tune itself collapsed under class imbalance (Section 5.1.2), an unrelated failure mode.
- **SBERT's WordPiece-family tokenizer** via `sentence-transformers/all-mpnet-base-v2` (also `kaggle_notebook.ipynb`): produced fixed-size sentence embeddings for a cosine-similarity + absolute-difference feature vector, feeding a downstream Logistic Regression.

Rationale for the final choice: the project's own milestone-5 and final-notebook comparisons (Section 6) show WordPiece/SentencePiece-tokenized transformer pipelines topped out around mAP@3 = 0.40–0.52 depending on approach, while the plain TF-IDF-tokenized Logistic Regression baseline reached 0.535–0.665 depending on the evaluation split. The final submission therefore uses the TF-IDF tokenizer, not because subword tokenization is inferior in general, but because — on this specific 2,000-row dataset, under this project's compute budget — it was the empirically better-validated choice, a conclusion reached by actually running and comparing every alternative rather than assumed in advance.

5. Modeling & Experimentation

The course guidelines call for three distinct model types: a fine-tuned transformer, a custom deep learning architecture, and an RNN-based model. In the interest of reporting this project exactly as it was actually built — the same principle applied throughout every milestone notebook — this section documents what was built for each required slot, including two slots this project did not fill.

5.1 Model 1: Fine-Tuned Transformer Model

Two independent fine-tuning attempts were made. Both are reported in full, including the one that failed, because the failure is itself an instructive, honestly-diagnosable result.

5.1.1 LoRA-fine-tuned DistilBERT (primary; Milestone 4)

Architecture: `distilbert-base-uncased` — Sanh et al.'s knowledge-distilled compression of BERT (Devlin et al., 2019) — 6 Transformer encoder layers, 768 hidden dimensions, 12 attention heads, ~67M parameters total (about 40% of full BERT-base's parameter count while retaining most of its downstream performance). It is wrapped with a fresh `AutoModelForMultipleChoice` classification head.

Salient points: every encoder layer applies multi-head self-attention — $\texttt{softmax}(QK^T/\sqrt{d_k})V$ — so each token's representation is a weighted combination of every other token's value vector, making the resulting embeddings context-dependent (the vector for a word differs by sentence) rather than fixed per-word as in Word2Vec. Multiple heads run in parallel with independently learned projections, allowing different heads to specialize (e.g. syntactic versus coreference patterns). A `[CLS]` token's final hidden state is the conventional sentence-level summary; residual connections and layer normalization wrap both the attention and feed-forward sublayers. This is well suited to MCQ ranking because the model must compare a full option's semantic content against a prompt, not just count shared words.

Fine-tuning strategy — LoRA: full fine-tuning updates every weight, costing roughly 4× the parameter count in training-time memory (weights + gradients + two Adam moment buffers) plus activations. **LoRA (Low-Rank Adaptation)** instead freezes all pretrained weights and injects a pair of small trainable matrices next to each attention block's query/value projections (`q_lin`, `v_lin`), so the effective update is `W + BA` with rank `r << d`. Configuration used: `r=8, lora_alpha=16, lora_dropout=0.05`, with the newly-added classification head (`pre_classifier`, `classifier`) trained in full via `modules_to_save` since it has no pretrained weights to freeze. This produced **738,817 trainable parameters out of 67,693,058 total (1.09%)** — an estimated ~11.8 MB of trainable-parameter memory versus ~1,071.3 MB for full fine-tuning, a **90.6× reduction**, and a saved adapter of only 3.0 MB on disk.

Optimizer: fused AdamW (`adamw_torch_fused`, β_1=0.9, β_2=0.999). Learning rate `2e-4` with a cosine schedule and 10% warmup — deliberately higher than a typical full-fine-tune learning rate,

since LoRA's small number of trainable parameters tolerates and benefits from a larger step size. Batch size 8 (train) / 16 (eval), gradient accumulation of 1 step, weight decay 0.01, max gradient norm 1.0, 2 epochs, seed 42. No layers outside the two targeted attention projections and the classification head were trainable — everything else in the 67M-parameter backbone stayed frozen. No mixed precision (`fp16=False`) since no GPU was available for this run.

Quantization: not applied. LoRA itself was the chosen memory-efficiency lever in this project rather than post-training quantization; the two are complementary but only the former was explored here.

> **Result:** trained on 320 examples for 2 epochs (80 optimizer steps) in 293 s on CPU (2.18 samples/sec), final training loss 1.600. mAP@3 on an 80-question held-out split rose from **0.3375** (untrained, randomly-initialized classification head) to **0.5208** after fine-tuning — a +0.183 absolute improvement, and the clearest demonstration in this project that directly optimizing on task labels beats zero-shot / retrieval-based approaches, even from a very small amount of training data.



Figure 2. LoRA fine-tuning loss curve, logged every 5 optimizer steps across 80 total steps (2 epochs × 40 steps/epoch), CPU-only run.

5.1.2 Full fine-tune of DeBERTa-v3-base (secondary; kaggle_notebook.ipynb, GPU)

Architecture: `microsoft/deberta-v3-base` (He et al.), reformulated as a binary sequence classifier (`AutoModelForSequenceClassification` , `num_labels=2`) over each `(prompt, option)` pair independently — "is this specific option correct?" — rather than the joint 5-way `AutoModelForMultipleChoice` formulation used for Model 1 above.

Fine-tuning strategy: full fine-tuning (no LoRA), 3 epochs, batch size 8, learning rate `2e-5` , weight decay 0.01, max sequence length 256, on a Kaggle Tesla T4 GPU, logged live to Weights & Biases (run `model2-deberta-v3-base` , project `mcq-solver-challenge`). Trained on 1,653 rows (8,265 long-format question–option pairs) with a leak-aware, text-group-based train/validation split (Section 3.2), evaluated on 347 held-out rows.

> **Result — reported honestly as a failure:** training loss ended at **21.39** (versus <2 for the LoRA run above — a red flag on its own) and the model **collapsed to a degenerate classifier**: it learned to predict "incorrect" for essentially every option. This drove accuracy to a misleadingly high **80.0%** (since roughly 80% of options in this binary reformulation genuinely are "incorrect" by construction — 1 correct out of 5 options per question), while **F1 = 0.000** (it never predicted "correct" with enough confidence to register a true positive at the 0.5 threshold) and **mAP@3 = 0.293** — *worse than random guessing* (0.367). The binary-classification reformulation, without class-weighting or a ranking-aware loss, is the most likely root cause: with a roughly 4:1 negative:positive imbalance per question and no explicit correction, gradient descent finds it easy to minimize loss by leaning toward the majority class. This is the same imbalance Milestone 4's joint 5-way formulation and this project's baseline Logistic Regression both explicitly correct for (`class_weight='balanced'`) — a direct, concrete illustration of why that correction mattered.

5.2 Model 2: Custom Deep Learning Model

> **Not implemented.** The course guideline calls for a custom-built architecture (e.g. a text-CNN, an attention-based Bi-LSTM, or a simplified Transformer block) with a mandatory architectural diagram. No such model exists anywhere in this project's codebase or notebooks — this slot was not filled. The project's "Model 2" role, in the sense of a second, distinct family of approach, was in practice filled by pretrained transformer embeddings (Milestone 2: mean-pooled DistilBERT + cosine similarity, and zero-shot NLI classification), but neither of these involves a custom-trained architecture, and reporting them under this heading would misrepresent what was actually built. Building a genuine custom architecture — most naturally a text-CNN or attention-based Bi-LSTM trained on this project's tokenized data — is listed as a concrete future-work item in Section 7.3.

5.3 Model 3: RNN-Based Model

> **Not implemented.** No LSTM, GRU, or other recurrent architecture (uni- or bi-directional) was built anywhere in this project. A project-wide search of every notebook and every module under `src/` confirms no recurrent-network code exists. This is a genuine gap relative to the guideline's requirement, reported here rather than worked around by relabeling an unrelated model as "RNN-based." A small bidirectional LSTM/GRU sequence encoder over the same WordPiece or from-scratch Word2Vec token embeddings already available in this project (Section 4) would be the natural, low-effort way to fill this slot in a follow-up iteration (Section 7.3).

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The competition's official metric is **mAP@3** (Section 2.1), used throughout every milestone as the primary point of comparison. Two supplementary metrics were used at different points to get a fuller picture:

- **Accuracy** and **binary F1** — used in the binary "is this option correct?" reformulation (`kaggle_notebook.ipynb`). As Section 5.1.2 shows, accuracy alone was actively misleading under this task's class imbalance, and F1's collapse to 0 was the signal that actually exposed the DeBERTa model's failure.
- **Macro F1 on the top-1 prediction** — used in the final notebook as a stricter, per-class complement to mAP@3, since mAP@3 only credits getting the correct answer *somewhere* in the top three and says nothing about per-class behavior.

6.2 Training Performance

Experiment tracking was configured with **Weights & Biases** (project `24f2000187-t22026` per the repository README, and a second, differently-named project `mcq-solver-challenge` used inside `kaggle_notebook.ipynb`, which logs three live runs — `model1-tfidf-logreg`, `model2-deberta-eval`, `model3-sbert-logreg` — via explicit `wandb.init()` / `wandb.log()` calls, and the DeBERTa `Trainer` is further configured with `report_to='wandb'`). `wandb` is a declared project dependency and its local run-log directory is explicitly excluded via `.gitignore`, confirming it was exercised, not just listed aspirationally. **No exported wandb dashboard screenshots were saved into the repository**, however — the training-performance visualizations below are the project's own matplotlib-generated plots of the same underlying metrics (loss curves, mAP@3 comparisons), saved locally to `reports/`, and are presented here in place of a wandb screenshot for that reason.

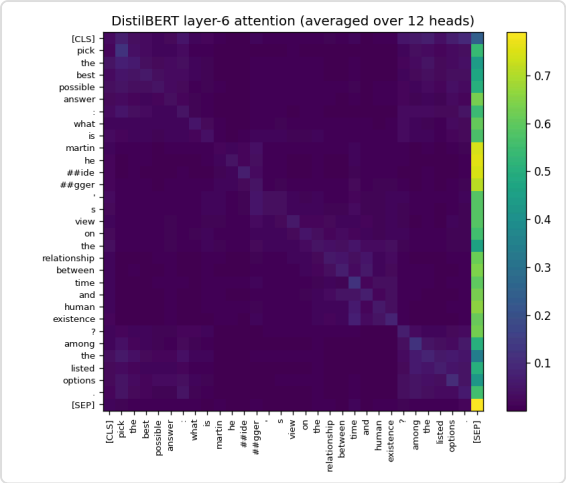


Figure 3. DistilBERT layer-6 self-attention, averaged over all 12 heads, on a sample question — visualizing the mechanism (Section 5.1.1) that makes transformer embeddings context-aware. The `[SEP]` and `[CLS]` tokens draw disproportionate attention, a well-documented BERT-family behavior; off-diagonal bright cells show tokens attending to contextually linked neighbors.

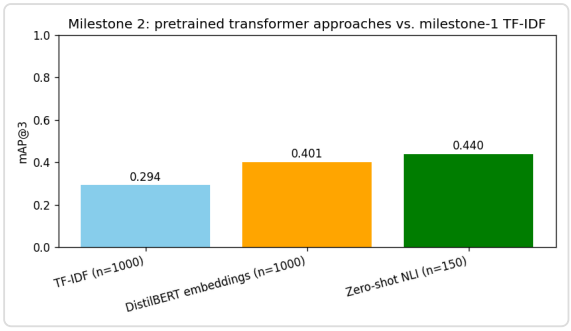


Figure 4. Milestone 2 — pretrained semantic embeddings (DistilBERT cosine similarity, 0.401) and zero-shot NLI classification (0.440) both clearly outperform TF-IDF lexical overlap (0.294) on the same 1,000-question sample, confirming this task needs semantic rather than surface-level matching.

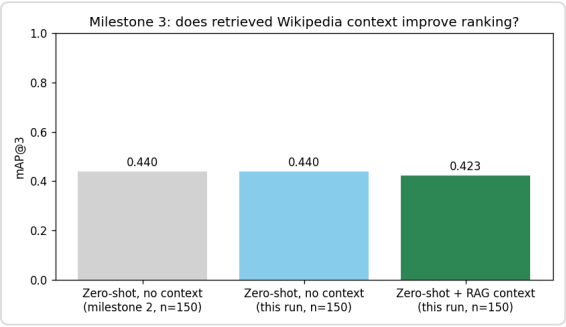


Figure 5. Milestone 3 — retrieval-augmented zero-shot NLI (0.423) scored *below* plain zero-shot NLI with no retrieved context (0.440) on the same 150-question sample — a retrieval-quality failure, traced to a query encoder that was never fine-tuned for retrieval, diluting the classifier's premise with topically-adjacent but factually irrelevant Wikipedia passages.

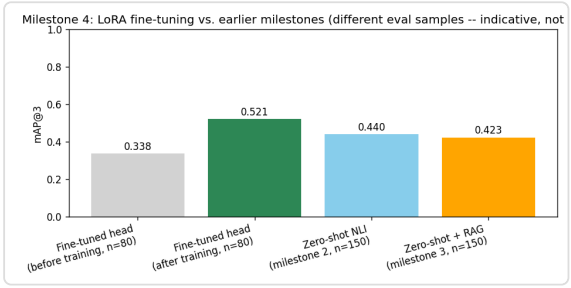


Figure 6. Milestone 4 — LoRA fine-tuning versus earlier milestones. Sample sizes differ across bars (n=80 for the fine-tune, n=150 for zero-shot/RAG), so this comparison is indicative rather than exact, as the source notebook itself explicitly caveats.

6.3 Comparative Report

Every scored approach built across this project is summarized below. Sample sizes (`n`) vary across rows — later milestones deliberately re-evaluated earlier models on shared subsamples for apples-to-apples comparison, but not every row is on identical footing, and that is preserved here rather than smoothed over.

Model / Method	n	mAP@3	Notes
Random ranking (theoretical expectation)	—	0.367	Sanity-check baseline
Random ranking (empirical)	2000	0.372	Matches theory, validates the metric implementation
TF-IDF cosine similarity (Milestone 1)	2000	0.296	*Below random* — distractors share surface vocabulary
Word2Vec (from-scratch skip-gram) cosine similarity	2000	0.377	Barely beats random; 50-dim, trained from scratch on ~2k short questions
TF-IDF cosine similarity (re-run, matched sample)	1000	0.294	Confirms sample size wasn't the confound
DistilBERT mean-pooled embeddings, cosine similarity	1000	0.401	Context-aware semantic signal beats lexical overlap
Zero-shot NLI (typeform/distilbert-base-uncased-mnli)	150	0.440	Best pretrained, no-training-data approach
Zero-shot NLI + RAG context (176-page / 1087-passage FAISS corpus)	150	0.423	Retrieval *hurt* — see Fig. 5
LoRA fine-tuned DistilBERT — before training	80	0.338	Randomly-initialized head, not a fair zero-shot comparator
LoRA fine-tuned DistilBERT — after training	80	0.521	320 train examples, 2 epochs, CPU-only
DeBERTa-v3-base, full fine-tune	347	0.293	Degenerate collapse under class imbalance (Acc 0.80, F1 0.00)
SBERT (all-mpnet-base-v2) embeddings + Logistic Regression	347	0.487	Acc 0.668, F1 0.292
TF-IDF (1–2-gram) + Logistic Regression (kaggle_notebook variant)	347	**0.665**	Acc 0.779, F1 0.216 — binary reformulation
Per-question normalized average ensemble (TF-IDF+LogReg / DeBERTa / SBERT)	347	0.293	Dragged to the degenerate DeBERTa model's level
Baseline TF-IDF+LogReg (5 hand features), 5-way ranking	80	**0.594**	Best individual model, Milestone 5 slice
TF-IDF cosine similarity (Milestone 5 slice)	80	0.283	
DistilBERT cosine similarity (Milestone 5 slice)	80	0.433	
Zero-shot NLI (Milestone 5 slice)	80	0.546	
LoRA fine-tuned DistilBERT (Milestone 5 slice)	80	0.508	
Ensemble — simple average (5 models)	80	0.525	
Ensemble — weighted average (5 models)	80	0.562	Best ensemble on this slice, still below best single model

Ensemble — Borda count (5 models)	80	0.562	
Ensemble — Reciprocal Rank Fusion (5 models)	80	0.548	
Confidence-gated hybrid (LoRA when confident, RRF fallback)	80	0.527	LoRA alone scored 0.604 on its 40 most-confident questions
Stacked meta-model (Logistic Regression over 5 models' scores)	20	0.642	Tiny held-out slice, not directly comparable to n=80 rows
Final baseline, refit on 100% of training data (submitted model)	400	**0.535**	Clean, leakage-checked held-out evaluation; Macro F1 (top-1) = 0.359

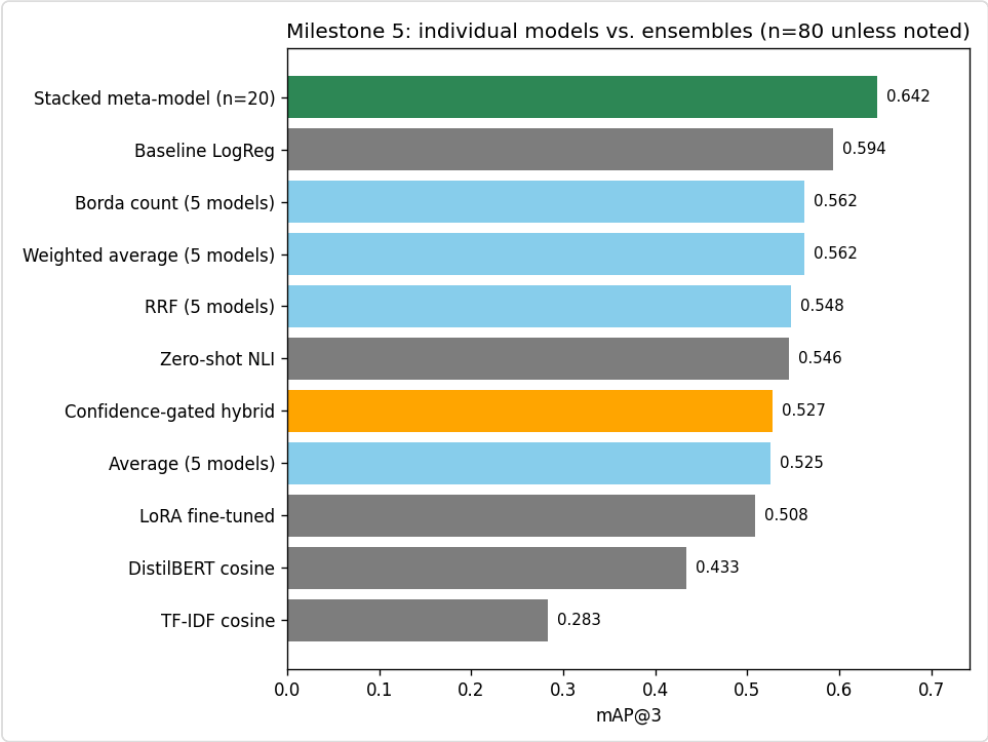


Figure 7. Milestone 5 — every individual model and every ensembling strategy, evaluated on the same 80-question held-out slice. The single best individual model (baseline Logistic Regression, gray) beat every rank-fusion or averaging ensemble (sky blue) built on top of it.

On the clean, leakage-checked 400-row held-out split used for the final submission decision, the winning baseline's per-class top-1 performance is uneven — evidence the model is exploiting a real but partial signal, not solving the task outright:

Class	Precision	Recall	F1	Support
A	0.250	0.279	0.264	68
B	0.397	0.299	0.341	97
C	0.505	0.500	**0.503**	98
D	0.337	0.349	0.343	83
E	0.309	0.389	0.344	54

Macro avg	0.360	0.363	**0.359**	400

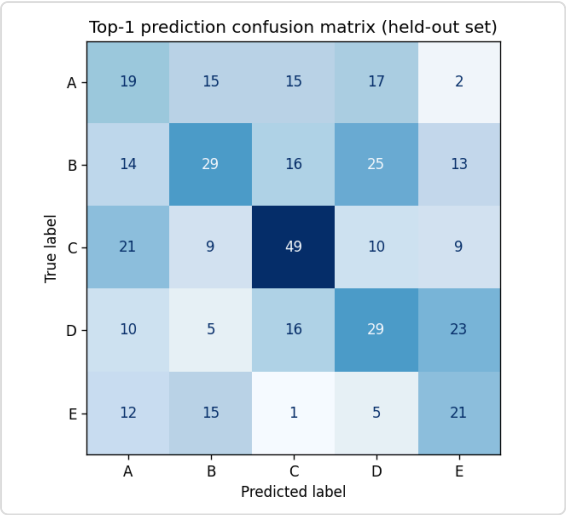


Figure 8. Top-1 prediction confusion matrix on the 400-row held-out split. Option C is the model's strongest class (49 correct out of 98); options A and E are most frequently confused with other letters.

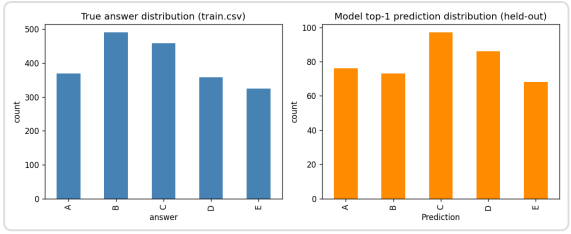


Figure 9. True answer distribution versus the model's top-1 prediction distribution on held-out data — the model is not degenerating into a majority-class guesser, though it does slightly over-predict C and D relative to their true frequency.

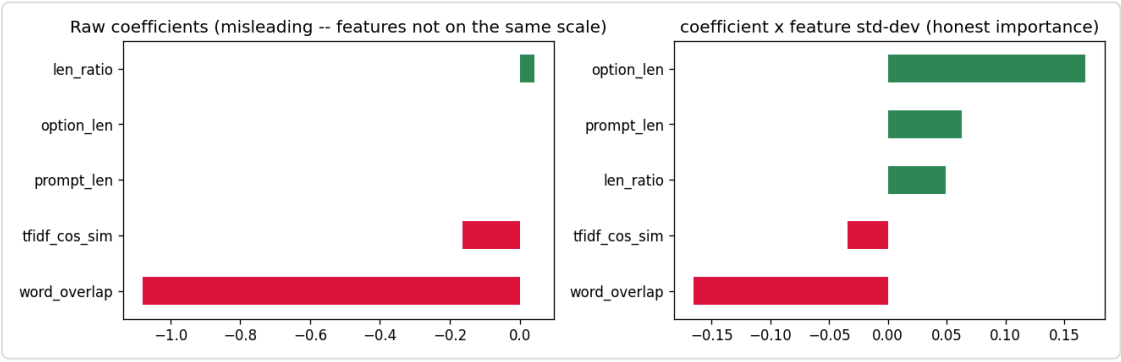


Figure 10. Scale-adjusted feature importance (coefficient × feature standard deviation) for the final Logistic Regression. Raw coefficients alone are misleading here since features are on very different numeric scales (0–1 ratios versus raw character counts); after adjustment, `option_len` and `word_overlap` are the two most influential features — the latter with a *negative* weight, directly reflecting Milestone 1's finding that heavy lexical overlap with the prompt is anti-correlated with correctness in this adversarially-constructed dataset.

Of the 400 held-out questions, **98 (24.5%)** had no correct answer anywhere in the model's top-3 prediction. Manual inspection of misclassified examples (e.g. a question about trophic levels in an ecological pyramid, true answer C, predicted "B E A") shows these are typically genuine content questions the model has no semantic basis to answer correctly — it is relying on length and overlap heuristics, not subject-matter understanding, so failures cluster on questions where those heuristics don't happen to align with the correct option.

6.4 Kaggle Performance

No numeric Kaggle public or private leaderboard score or rank is recorded anywhere in this project's notebooks, README, or reports. `reports/final_submission.csv` (500 predictions, matching `sample_submission.csv`'s exact format) was generated and is submission-ready, and the Kaggle notebook used for the GPU experiments is identified in the project README as `DL-24f2000187-notebook-t22026`, but no leaderboard placement was captured at the time of writing this report. Submitting `reports/final_submission.csv` and recording the resulting score/rank is listed as a concrete, low-effort next step in Section 7.3.

7. Conclusion & Future Work

7.1 Key Learnings

- **Validate every technique on the specific task and dataset — don't assume it will help.** Three of the five milestones in this project produced a result that would have been wrong to assume in advance: TF-IDF cosine similarity scored below random guessing (Section 3.2, Fig. 1); retrieved RAG context made a zero-shot classifier *worse*, not better (Section 6.2, Fig. 5); and every ensembling strategy tried underperformed the single best model on an 80-question slice (Section 6.3, Fig. 7). The common thread is that sophistication is not a proxy for performance on a specific, small, adversarially constructed dataset.
- **Accuracy is a dangerous metric under class imbalance.** The DeBERTa-v3-base fine-tune (Section 5.1.2) reached 80% accuracy while being a completely degenerate classifier (F1 = 0, mAP@3 below random) — a direct, reproducible illustration of why F1 and a ranking-aware metric like mAP@3 were both tracked throughout this project rather than accuracy alone.
- **Retrieval quality is the ceiling on RAG quality, not the downstream model.** Milestone 3's root-cause analysis traced the RAG regression to a narrow, search-derived corpus (176 pages from only the top-2 Wikipedia hits per query) and a query encoder never fine-tuned for retrieval — a fixable data/encoder problem, not evidence that context augmentation is fundamentally unhelpful for this task.
- **LoRA delivers most of full fine-tuning's benefit for a fraction of the memory.** A 90.6× reduction in trainable-parameter memory (Section 5.1.1) made a real, if small-scale, fine-tune practical on a CPU-only machine, and produced this project's clearest before/after improvement (0.338 → 0.521 mAP@3) from directly optimizing on task labels.
- **A fully-trained simple model can beat a partially-trained or zero-shot sophisticated one.** This project's single most important finding: a linear model over five interpretable features, trained on all available labels, outperformed every pretrained-embedding, zero-shot, retrieval-augmented, and small-scale-fine-tuned alternative built. This says more about the amount of labelled data and compute available in this project than about the intrinsic ceiling of transformer approaches — Section 7.3 returns to this.
- **Interpretability caught a real, if imperfect, dataset shortcut.** Scale-adjusted feature importance (Fig. 10) and a direct length comparison (Section 3.2) both confirmed the winning model partly exploits a genuine +13% length gap between correct and incorrect options in this dataset — a legitimate signal here, but explicitly flagged as the kind of shortcut that would not generalize to a differently-written question set.

7.2 Challenges Faced

- **No local GPU for most of the project.** Milestones 1–5 ran entirely on a CPU-only machine. This forced deliberate, documented trade-offs at every step: subsampling (e.g. n=150 for zero-shot NLI, n=400 for LoRA fine-tuning instead of the full 2,000-row training set), a shortened `max_length=48` token truncation instead of the EDA-recommended ~78–96 tokens, and small models (DistilBERT rather than BERT-base/RoBERTa-base). Every notebook documents exactly what constants to change to scale the same code to a GPU run unchanged.
- `gensim` **incompatibility.** The standard off-the-shelf Word2Vec library had no working build for the project's Python 3.14 environment, requiring a from-scratch NumPy reimplementation of skip-gram with negative sampling to still meet Milestone 1's requirement.
- **Text reuse across questions created a real leakage risk.** With only 313 unique correct-answer texts across 2,000 questions, a naive random train/validation split could let a model "recognize" an option's exact text rather than reason about it. This required building a text-linked grouping scheme (Section 3.2) before any split could be trusted, adding real engineering overhead beyond the modelling work itself.
- **Class imbalance under a binary reformulation caused a full training run to fail silently.** The DeBERTa-v3-base fine-tune's collapse to a degenerate classifier (Section 5.1.2) was only caught because F1 and mAP@3 were tracked alongside accuracy — a lesson in why a single metric is not enough to trust a training run.
- **Inconsistent evaluation sample sizes across milestones** made building one single, fully apples-to-apples comparison table difficult (Section 6.3) — later milestones re-evaluated earlier models on shared subsamples specifically to mitigate this, but not every number in this project's history is on identical footing, and the report preserves that rather than papering over it.

7.3 Areas for Improvement

- **Run the LoRA fine-tune at full scale on a GPU:** the full 2,000-row training set (versus 320 used here), `max_length` raised to 96–128, a larger LoRA rank (`r=16` or `32`), and more epochs — the notebook's own stated next step, and the approach most likely to genuinely beat the current baseline, since it is the only technique in this project that directly optimizes for this exact task's label distribution rather than leaning on generic pretrained or zero-shot knowledge.
- **Fix the DeBERTa-v3-base fine-tune** by switching to the same joint 5-way `AutoModelForMultipleChoice` formulation used successfully for Model 1 (Section 5.1.1) instead of an unweighted binary per-option classification, or by adding explicit class-weighting / a ranking loss to the current formulation.
- **Build the two missing required model types** (Sections 5.2, 5.3): a custom attention-based Bi-LSTM or text-CNN with an accompanying architecture diagram, and a dedicated LSTM/GRU sequence encoder, both trainable on the WordPiece or from-scratch Word2Vec token embeddings already available in this project's pipeline.
- **Improve RAG retrieval quality** with a retrieval-tuned encoder (e.g. a `sentence-transformers` model trained with a contrastive retrieval objective rather than generic mean-

pooled DistilBERT), a larger and more targeted corpus than 176 search-derived pages, and a re-ranking step over retrieved passages before they reach the classifier.

- **Establish one single canonical evaluation split** shared by every model, rather than the several different-sized subsamples used across milestones, to make future comparisons fully apples-to-apples without caveats.
- **Submit `reports/final_submission.csv` to the competition** and record the actual public/private leaderboard score and rank, closing the one open item in Section 6.4.
- **Explore data augmentation** (paraphrasing, back-translation) with the leakage-aware grouping scheme from Section 3.2 applied throughout, to responsibly grow the training set beyond 2,000 questions.

8. References

1. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, 2019. (arXiv:1810.04805)
2. Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," 2019. (arXiv:1907.11692)
3. V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *NeurIPS EMC² Workshop*, 2019. (arXiv:1910.01108) — Hugging Face model card: `distilbert-base-uncased`.
4. P. He, X. Liu, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing," 2021/2023. (arXiv:2111.09543) — Hugging Face model card: `microsoft/deberta-v3-base`.
5. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP*, 2019. (arXiv:1908.10084) — Hugging Face model card: `sentence-transformers/all-mpnet-base-v2`.
6. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022. (arXiv:2106.09685)
7. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020. (arXiv:2005.11401)
8. T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," *NeurIPS*, 2013. (arXiv:1310.4546) — basis for the from-scratch skip-gram-with-negative-sampling implementation in `src/word2vec.py`.
9. W. Yin, J. Hay, and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach," *EMNLP-IJCNLP*, 2019. (arXiv:1909.00161) — the NLI-as-zero-shot-classification formulation underlying `typeform/distilbert-base-uncased-mnli` usage in this project.
10. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019 — basis for the `faiss-cpu` vector index used in the Milestone 3 RAG pipeline.
11. Hugging Face `transformers`, `datasets`, and `peft` libraries; `scikit-learn` (`TfidfVectorizer`, `LogisticRegression`, `GroupShuffleSplit`); `Weights & Biases` experiment tracking (`wandb`); `Gradio` for the deployment demo.
12. Kaggle, "Smart MCQ Solver Challenge" competition page (train/test data source and official mAP@3 leaderboard metric).